

Portada

Título:

Diseño Arquitectónico – Sistema de Banca por Internet BP

Subtítulo:

Evaluación Técnica – Arquitecto de Soluciones / Líder Técnico de Desarrollo

Autor:

David Palacios

Fecha: 10 De Junio Del 2026

Versión 1.1.1

1. Introducción

1.1 Objetivo del documento

El presente documento tiene como objetivo definir la arquitectura de solución para la plataforma de Banca por Internet de la entidad BP, proporcionando una visión integral de los componentes funcionales, tecnológicos y operativos necesarios para soportar los servicios digitales ofrecidos a sus clientes.

La propuesta arquitectónica busca satisfacer los requerimientos funcionales planteados para consulta de movimientos, transferencias, pagos y onboarding digital, así como los requerimientos no funcionales asociados a disponibilidad, seguridad, resiliencia, trazabilidad, cumplimiento regulatorio y escalabilidad.

Adicionalmente, este documento presenta el diseño arquitectónico siguiendo el modelo C4 (Contexto, Contenedores y Componentes), incorporando principios de arquitectura moderna basados en microservicios, eventos, observabilidad y seguridad por diseño (Security by Design).

1.2 Alcance

La arquitectura propuesta cubre los siguientes dominios funcionales:

- Consulta de información de clientes.
- Consulta de productos bancarios.
- Consulta de movimientos históricos.
- Transferencias entre cuentas propias.
- Transferencias interbancarias.
- Pago de servicios y terceros.
- Onboarding digital con validación biométrica.
- Gestión de autenticación y autorización.
- Gestión de auditoría y trazabilidad.
- Gestión de notificaciones regulatorias.
- Integración con sistemas internos y externos.

La propuesta contempla además componentes de infraestructura cloud, mecanismos de alta disponibilidad, recuperación ante desastres, monitoreo, observabilidad y controles de seguridad requeridos para una entidad financiera moderna.

1.3 Supuestos

Para el diseño de la solución se consideran los siguientes supuestos:

- La entidad BP dispone de un Core Bancario que expone servicios de integración para consulta y ejecución de transacciones.
- Existe un sistema complementario que provee información detallada de clientes.
- La organización cuenta con un proveedor de identidad configurable compatible con OAuth 2.0 y OpenID Connect.
- Existe presupuesto suficiente para utilizar servicios administrados en la nube.
- Las integraciones con ACH, BCE y pasarelas de pago disponen de interfaces de comunicación seguras y documentadas.
- Los canales digitales serán utilizados por clientes de banca personal inicialmente, con posibilidad de expansión futura hacia banca empresarial.

1.4 Restricciones

Las siguientes restricciones fueron identificadas a partir del enunciado:

- Debe implementarse el modelo C4 hasta nivel de Componentes.
- Deben existir al menos dos mecanismos de notificación.
- Debe utilizarse OAuth 2.0 como mecanismo principal de autenticación.
- Debe contemplarse onboarding con reconocimiento facial.
- Debe existir un mecanismo de auditoría integral.
- Debe garantizarse alta disponibilidad, resiliencia y recuperación ante desastres.
- La solución debe ser escalable y preparada para el crecimiento futuro.

1.5 Audiencia objetivo

Este documento está dirigido a:

- Arquitectos de Solución.
- Arquitectos Empresariales.
- Arquitectos Cloud.
- Líderes Técnicos.
- Product Owners.
- Equipos de Desarrollo.
- Equipos de Seguridad.
- Equipos de Infraestructura.
- Auditores internos y externos.
- Stakeholders de negocio.

2. Análisis del Problema

2.1 Contexto de Negocio

La entidad financiera BP requiere implementar una plataforma de Banca por Internet moderna que permita a sus clientes realizar operaciones financieras a través de canales digitales web y móviles.

Actualmente la información necesaria para atender las operaciones bancarias se encuentra distribuida entre múltiples sistemas empresariales. El sistema principal corresponde al Core Bancario, encargado de gestionar clientes, cuentas, productos y movimientos financieros, mientras que un sistema complementario provee información adicional requerida para determinados procesos de consulta y validación.

La solución propuesta debe ofrecer una experiencia digital segura, consistente y omnicanal, permitiendo a los clientes realizar consultas, transferencias, pagos y procesos de onboarding digital sin necesidad de acudir a una sucursal física.

Desde la perspectiva empresarial, la plataforma deberá convertirse en un habilitador estratégico para la transformación digital de la institución financiera, reduciendo costos operativos, incrementando la adopción de canales digitales y fortaleciendo los mecanismos de seguridad, trazabilidad y cumplimiento regulatorio.

2.2 Objetivos de Negocio

La arquitectura propuesta debe contribuir al cumplimiento de los siguientes objetivos estratégicos:

ID	Objetivo
OBJ-01	Permitir a los clientes consultar información financiera desde cualquier canal digital.
OBJ-02	Habilitar transferencias entre cuentas propias y cuentas de terceros.
OBJ-03	Facilitar transferencias interbancarias mediante integración con redes financieras externas.
OBJ-04	Permitir pagos digitales a servicios, comercios y terceros.
OBJ-05	Incorporar nuevos clientes mediante procesos de onboarding digital.
OBJ-06	Reducir la dependencia de canales presenciales.
OBJ-07	Incrementar la satisfacción y experiencia del cliente.
OBJ-08	Garantizar trazabilidad completa de todas las operaciones.

OBJ-09	Cumplir normativas financieras y regulatorias.
OBJ-10	Disponer de una arquitectura escalable preparada para futuros productos digitales.

2.3 Actores del Sistema

- **Cliente Existente**

Persona que posee productos financieros activos y utiliza la plataforma para consultar información, realizar transferencias y efectuar pagos.

- **Cliente Nuevo**

Persona que aún no posee acceso a la plataforma y requiere completar un proceso de onboarding digital con validación biométrica para habilitar sus credenciales.

- **Administrador Operativo**

Usuario interno responsable de monitorear el funcionamiento de la plataforma, revisar eventos de auditoría y gestionar incidencias operativas.

- **Equipo de Soporte**

Responsable de brindar soporte técnico, análisis de incidencias y atención de eventos críticos relacionados con la operación de la plataforma.

- **Sistemas Externos**

Incluyen el Core Bancario, sistema complementario de clientes, redes interbancarias, motores biométricos, proveedores de notificaciones y sistemas regulatorios.

2.4 Requerimientos Funcionales

- **Gestión de Clientes**

ID	Requerimiento
RF-01	Consultar información básica del cliente.
RF-02	Consultar productos financieros asociados al cliente.
RF-03	Consultar información ampliada proveniente del sistema complementario.

- **Consulta de Información Financiera**

ID	Requerimiento
RF-04	Consultar saldo de cuentas.
RF-05	Consultar movimientos históricos.
RF-06	Filtrar movimientos por rango de fechas.
RF-07	Consultar detalles de una transacción específica.

- **Transferencias**

ID	Requerimiento
RF-08	Realizar transferencias entre cuentas propias.
RF-09	Realizar transferencias entre cuentas del mismo banco.
RF-10	Realizar transferencias interbancarias.
RF-11	Validar saldo disponible antes de ejecutar una transferencia.
RF-12	Notificar el resultado de la transferencia al cliente.

- **Pagos**

ID	Requerimiento
RF-13	Realizar pagos a servicios.
RF-14	Realizar pagos a terceros.
RF-15	Confirmar y registrar pagos efectuados.

- **Autenticación y Seguridad**

ID	Requerimiento
RF-16	Permitir autenticación mediante usuario y contraseña.
RF-17	Permitir autenticación biométrica mediante huella o reconocimiento facial del dispositivo.
RF-18	Gestionar sesiones seguras mediante OAuth 2.0 y OpenID Connect.
RF-19	Aplicar autenticación reforzada para operaciones sensibles.

- **Onboarding Digital**

ID	Requerimiento
RF-20	Registrar nuevos clientes mediante onboarding digital.
RF-21	Validar identidad mediante reconocimiento facial.
RF-22	Capturar documentos de identidad.
RF-23	Realizar validaciones de liveness detection.
RF-24	Crear credenciales de acceso para nuevos usuarios.

- **Notificaciones**

ID	Requerimiento
RF-25	Enviar notificaciones por correo electrónico.
RF-26	Enviar notificaciones por SMS.
RF-27	Enviar notificaciones push a dispositivos móviles.
RF-28	Confirmar entrega de notificaciones.

- **Auditoría**

ID	Requerimiento
RF-29	Registrar todos los eventos de autenticación.
RF-30	Registrar consultas realizadas por los clientes.
RF-31	Registrar transferencias y pagos.
RF-32	Mantener trazabilidad completa de las operaciones.

2.5 Requerimientos No Funcionales

Categoría	Requerimiento
Disponibilidad	Disponibilidad mínima de 99.95%.
Rendimiento	Consultas inferiores a 300 ms para operaciones cacheadas.
Rendimiento	Operaciones transaccionales inferiores a 3 segundos.
Escalabilidad	Escalamiento horizontal de componentes críticos.
Resiliencia	Tolerancia a fallos sin interrupción del servicio.
Recuperación	RTO menor o igual a 1 hora.
Recuperación	RPO menor o igual a 15 minutos.
Seguridad	Cifrado TLS 1.3 en tránsito.
Seguridad	Cifrado AES-256 en reposo.
Seguridad	Autenticación multifactor.
Auditoría	Registro de todas las operaciones ejecutadas.
Observabilidad	Monitoreo centralizado de logs, métricas y trazas.
Operación	Despliegues automatizados mediante CI/CD.
Mantenibilidad	Arquitectura desacoplada y extensible.
Cumplimiento	Cumplimiento de PCI DSS, ISO 27001 y LOPDP.

2.6 Drivers Arquitectónicos

Los drivers arquitectónicos representan aquellos factores que influyen directamente sobre las decisiones de diseño.

- **Driver 1: Seguridad**

Debido a la naturaleza financiera de la solución, la seguridad se convierte en el principal driver arquitectónico. La plataforma deberá proteger información sensible, prevenir accesos no autorizados, garantizar autenticación robusta y mantener la integridad de las transacciones.

- **Driver 2: Disponibilidad**

Los servicios financieros digitales deben permanecer operativos de forma continua. Una interrupción afecta directamente la confianza del cliente y puede generar impactos económicos y regulatorios.

- **Driver 3: Escalabilidad**

La plataforma debe soportar incrementos significativos de usuarios, transacciones y productos digitales sin requerir rediseños arquitectónicos.

- **Driver 4: Cumplimiento Regulatorio**

La arquitectura debe incorporar controles que permitan cumplir con regulaciones financieras, requisitos de auditoría y protección de datos personales.

- **Driver 5: Integración**

La solución dependerá de múltiples sistemas internos y externos, por lo que deberá facilitar integraciones desacopladas y resilientes.

- **Driver 6: Observabilidad**

La operación bancaria exige monitoreo continuo, trazabilidad completa y capacidad de diagnóstico ante incidentes.

2.7 Riesgos Arquitectónicos Identificados

ID	Riesgo	Impacto
RA-01	Indisponibilidad del Core Bancario	Alto
RA-02	Saturación por crecimiento de usuarios	Alto
RA-03	Fallos en proveedores externos de notificaciones	Medio
RA-04	Fraude mediante robo de credenciales	Alto
RA-05	Fallos en integración con ACH o BCE	Alto
RA-06	Pérdida de eventos de auditoría	Crítico
RA-07	Exposición de información biométrica	Crítico
RA-08	Dependencia excesiva de componentes monolíticos	Alto
RA-09	Latencia excesiva en consultas al Core	Alto
RA-10	Incumplimiento regulatorio	Crítico

2.8 Quality Attribute Scenarios (QAS)

- **Escenario de Disponibilidad**

Cuando falle una instancia de un microservicio en producción, el sistema deberá recuperarse automáticamente sin intervención manual y sin afectar la continuidad del servicio para el usuario final.

- **Escenario de Rendimiento**

Cuando la plataforma reciba un incremento repentino del 300% en consultas de movimientos, el tiempo promedio de respuesta deberá mantenerse por debajo de un segundo mediante mecanismos de cacheo y escalamiento automático.

- **Escenario de Seguridad**

Cuando un usuario intente acceder a información sensible desde un dispositivo desconocido, el sistema deberá exigir autenticación reforzada mediante MFA antes de permitir la operación.

- **Escenario de Auditoría**

Cuando un auditor solicite el historial de una operación financiera, deberá ser posible reconstruir completamente la secuencia de eventos asociados a dicha operación.

- **Escenario de Recuperación**

Cuando ocurra una caída total de la región principal de nube, la plataforma deberá restablecer operaciones desde la región secundaria dentro del RTO y RPO definidos.

- **Escenario de Escalabilidad**

Cuando el volumen de usuarios concurrentes se duplique durante campañas comerciales o fechas de alta demanda, la plataforma deberá escalar horizontalmente sin afectar la experiencia del cliente.

3. Decisiones Arquitectónicas (ADR)

Las siguientes Architectural Decision Records (ADR) documentan las principales decisiones de diseño adoptadas para la solución propuesta.

Su objetivo es proporcionar trazabilidad arquitectónica, justificar la selección tecnológica realizada y facilitar futuras evoluciones de la plataforma.

ADR-001 - Arquitectura basada en Microservicios

Estado: Aceptado

Contexto:

La plataforma de banca digital deberá soportar múltiples dominios funcionales independientes, incluyendo autenticación, consulta de movimientos, transferencias, pagos, onboarding, auditoría y notificaciones.

Cada dominio posee patrones de consumo distintos, diferentes niveles de criticidad y requerimientos de escalabilidad particulares.

Además, la entidad financiera requiere una arquitectura preparada para incorporar nuevos productos digitales en el futuro sin afectar la estabilidad de los servicios existentes.

Alternativas Evaluadas

- Monolito tradicional.
- Monolito modular.
- Arquitectura orientada a servicios (SOA).
- Arquitectura basada en microservicios.

Decisión

Se adopta una arquitectura basada en microservicios.

Justificación

- **Justificación 1: Escalabilidad independiente por dominio**
Los diferentes procesos bancarios presentan comportamientos de carga distintos. Por ejemplo, las consultas de movimientos pueden representar decenas de veces más tráfico que las transferencias o pagos.
Una arquitectura de microservicios permite escalar únicamente los servicios que experimentan mayor demanda, optimizando recursos de infraestructura y reduciendo costos operativos.
- **Justificación 2: Aislamiento de fallos**
En sistemas financieros es fundamental evitar que la falla de un componente afecte al resto de la plataforma.
Mediante microservicios es posible aplicar mecanismos de resiliencia específicos por dominio (Circuit Breaker, Retry, Bulkhead), limitando la propagación de errores y mejorando la disponibilidad global del sistema.
- **Justificación 3: Evolución tecnológica independiente**
La plataforma deberá evolucionar durante varios años. Los microservicios permiten modificar, reemplazar o evolucionar componentes individuales sin requerir despliegues masivos ni afectar otros dominios de negocio.

Consecuencias Positivas

- Mayor escalabilidad.
- Mejor desacoplamiento.
- Mayor resiliencia.
- Despliegues independientes.

Consecuencias Negativas

- Mayor complejidad operativa.
- Requiere observabilidad avanzada.
- Incrementa la complejidad de pruebas distribuidas.

ADR-002 - Arquitectura Event-Driven

Estado: Aceptado

Contexto:

La plataforma debe procesar eventos de negocio como transferencias, pagos, autenticaciones, onboarding y auditoría. Adicionalmente, múltiples componentes necesitan reaccionar a los mismos eventos sin generar dependencias directas.

Alternativas Evaluadas:

- Comunicación exclusivamente síncrona REST.
- RabbitMQ.
- AWS SNS/SQS.
- Arquitectura basada en eventos con Kafka.

Decisión

Adoptar una arquitectura Event-Driven soportada por Apache Kafka (AWS MSK).

Justificación

- **Justificación 1: Desacoplamiento entre dominios**

Cuando una transferencia es completada, múltiples sistemas necesitan reaccionar:

- Auditoría.
- Notificaciones.
- Monitoreo.
- Analítica.

Kafka permite que cada consumidor procese el evento de forma independiente sin afectar al productor original.

- **Justificación 2: Persistencia y trazabilidad de eventos**

A diferencia de mecanismos tradicionales de mensajería, Kafka conserva los eventos durante períodos configurables. Esto facilita auditorías regulatorias, reconstrucción de operaciones y análisis posteriores.

- **Justificación 3: Escalabilidad horizontal**

Kafka permite distribuir cargas entre múltiples particiones y consumidores, soportando altos volúmenes transaccionales sin afectar la estabilidad de la plataforma.

Consecuencias Positivas:

- Desacoplamiento.
- Escalabilidad.
- Trazabilidad.

Consecuencias Negativas

- Mayor complejidad operativa.
- Curva de aprendizaje superior.
- Necesidad de gobernanza de eventos.

ADR-003 - AWS como Plataforma Cloud Principal

Estado: Aceptado

Contexto:

La solución requiere alta disponibilidad, recuperación ante desastres, escalabilidad automática, servicios administrados y cumplimiento regulatorio.

Alternativas Evaluadas

- Infraestructura On-Premise.
- Microsoft Azure.
- Google Cloud Platform.
- Amazon Web Services.

Decisión: Adoptar AWS como plataforma cloud principal.

Justificación

- **Justificación 1: Madurez en el sector financiero**
AWS posee amplia adopción en instituciones financieras globales y regionales, además de certificaciones como PCI DSS, ISO 27001, SOC 1, SOC 2 y SOC 3. Esto reduce riesgos de cumplimiento y facilita auditorías regulatorias.
- **Justificación 2: Amplio catálogo de servicios administrados**
Servicios como EKS, RDS, MSK, ElastiCache y CloudWatch permiten reducir significativamente la carga operativa sobre equipos de infraestructura. Esto permite concentrar esfuerzos en capacidades de negocio y no en administración de plataformas.
- **Justificación 3: Alta disponibilidad nativa**
AWS proporciona capacidades Multi-AZ, Auto Scaling y recuperación automatizada, fundamentales para alcanzar los objetivos de disponibilidad definidos para la plataforma.

Consecuencias Positivas

- Menor operación.
- Alta disponibilidad.
- Escalabilidad nativa.

Consecuencias Negativas

- Dependencia del proveedor.
- Costos de salida.
- Necesidad de gobierno cloud.

ADR-004 - API Gateway como Punto Único de Entrada

Estado: Aceptado

Alternativas Evaluadas

- Acceso directo a microservicios.
- Reverse Proxy tradicional.
- API Gateway.

Decisión: Implementar API Gateway como punto único de acceso.

Justificación

- **Justificación 1:** Centraliza autenticación, autorización, rate limiting y validación de tokens.
- **Justificación 2:** Reduce la exposición directa de microservicios a Internet.
- **Justificación 3:** Permite aplicar políticas de seguridad y observabilidad de forma uniforme en toda la plataforma.

Consecuencias Positivas

- Seguridad centralizada.
- Menor acoplamiento.
- Gobierno de APIs.

Consecuencias Negativas

- Posible cuello de botella si no se diseña adecuadamente.

ADR-005 - Patrón Backend For Frontend (BFF)

Decisión: Implementar un BFF para Web y un BFF para Mobile.

Justificación

- **Justificación 1:** Los canales web y móvil poseen necesidades distintas de presentación y agregación de información.
- **Justificación 2:** Permite optimizar payloads y reducir llamadas desde dispositivos móviles.
- **Justificación 3:** Evita que cambios en microservicios impacten directamente los canales digitales.

ADR-006 OAuth 2.0 + OpenID Connect

Decisión: Implementar OAuth 2.0 y OpenID Connect.

Justificación

- **Justificación 1:** OAuth 2.0 es el estándar predominante para autorización segura en aplicaciones modernas.
- **Justificación 2:** OpenID Connect extiende OAuth proporcionando identidad verificable mediante tokens estandarizados.
- **Justificación 3:** Permite integrar mecanismos avanzados como MFA, biometría y Passkeys sin modificar los sistemas consumidores.

ADR-007 - Authorization Code Flow con PKCE

Decisión: Utilizar Authorization Code Flow con PKCE para SPA y Mobile.

Justificación

- **Justificación 1:** Mitiga ataques de interceptación del código de autorización.
- **Justificación 2:** Elimina la necesidad de almacenar Client Secrets en aplicaciones públicas.
- **Justificación 3:** Es el flujo recomendado actualmente por OAuth 2.1 y OWASP ASVS.

ADR-008 - Keycloak como Identity Provider

Decisión: Utilizar Keycloak como proveedor de identidad.

Justificación

- **Justificación 1:** Implementa OAuth2, OIDC, SAML y FIDO2 de forma nativa.
- **Justificación 2:** Permite personalizar flujos de autenticación para onboarding biométrico.
- **Justificación 3:** Evita costos por usuario asociados a soluciones comerciales.

ADR-009 - Angular para la SPA

Decisión: Seleccionar Angular para el canal web.

Justificación

- **Justificación 1:**
Angular incorpora mecanismos nativos de protección contra Cross-Site Scripting (XSS) mediante sanitización automática del DOM, reduciendo significativamente la superficie de ataque frente a aplicaciones financieras expuestas a Internet.
- **Justificación 2:**
La arquitectura basada en módulos, componentes, servicios e inyección de dependencias favorece el mantenimiento a largo plazo y reduce la deuda técnica en proyectos empresariales complejos.
- **Justificación 3:**
Su integración nativa con TypeScript facilita la construcción de aplicaciones robustas, permitiendo validaciones tempranas, mejor mantenibilidad y menor probabilidad de errores en producción.

ADR-010 - React Native para Aplicación Móvil

Decisión: Seleccionar React Native como framework multiplataforma.

Justificación

- **Justificación 1:**
Permite reutilizar conocimientos y prácticas de desarrollo basadas en JavaScript y TypeScript, reduciendo tiempos de capacitación y facilitando la incorporación de nuevos desarrolladores.
- **Justificación 2:**
Ofrece acceso maduro a capacidades biométricas nativas mediante integración con Face ID, Touch ID y Android Biometric Authentication, fundamentales para procesos de autenticación bancaria.
- **Justificación 3:**
Reduce el costo total de propiedad al mantener una única base de código para iOS y Android, disminuyendo esfuerzos de mantenimiento y evolución funcional.

ADR-011 - PostgreSQL como Base de Datos Transaccional Principal

Estado: Aceptado

Contexto:

La plataforma de banca digital requiere almacenar información transaccional relacionada con transferencias, pagos, onboarding, auditoría operacional y configuraciones del sistema.

Los datos gestionados poseen características altamente transaccionales, requieren consistencia fuerte y deben garantizar integridad ante fallos parciales, errores de red o concurrencia elevada. Adicionalmente, la solución debe soportar mecanismos de replicación, recuperación ante desastres, cifrado y auditoría.

Alternativas Evaluadas

- Oracle Database.
- Microsoft SQL Server.
- MySQL.
- PostgreSQL.

Decisión: Adoptar PostgreSQL como motor principal para la persistencia transaccional.

Justificación

● **Justificación 1: Cumplimiento de propiedades ACID**

Las operaciones bancarias requieren garantizar atomicidad, consistencia, aislamiento y durabilidad.

PostgreSQL implementa completamente las propiedades ACID, permitiendo asegurar que operaciones críticas como transferencias, pagos y actualizaciones de saldo mantengan integridad incluso ante fallos de infraestructura o interrupciones inesperadas.

● **Justificación 2: Capacidades empresariales avanzadas sin costos de licenciamiento**

PostgreSQL ofrece replicación, particionamiento, cifrado, control granular de acceso, procedimientos almacenados y alta disponibilidad sin requerir licencias propietarias. Esto permite construir una solución empresarial robusta reduciendo costos operativos y dependencia tecnológica.

● **Justificación 3: Amplia adopción en entornos financieros**

PostgreSQL es ampliamente utilizado en instituciones financieras, fintechs y plataformas de pagos debido a su estabilidad, rendimiento y madurez.

Además, cuenta con soporte nativo en AWS RDS, facilitando operaciones de backup, recuperación, monitoreo y escalabilidad.

Consecuencias Positivas

- Consistencia transaccional.
- Alta confiabilidad.

- Menor costo de licenciamiento.
- Compatibilidad con servicios administrados.

Consecuencias Negativas

- Escalabilidad horizontal más compleja que bases NoSQL.
- Requiere estrategias específicas para particionamiento y crecimiento masivo.

ADR-012 - Redis y Patrón Cache-Aside para Optimización de Consultas

Estado: Aceptado

Contexto:

La mayoría de las operaciones realizadas por los clientes corresponden a consultas de información como:

- Productos.
- Perfil.
- Saldos.
- Movimientos recientes.

Estas consultas generan una carga significativa sobre los sistemas Core, los cuales normalmente poseen costos elevados de procesamiento y tiempos de respuesta superiores a los requeridos por los canales digitales.

Alternativas Evaluadas

- Sin mecanismo de caché.
- Caché local en aplicación.
- Caché distribuida Redis.
- Base de datos NoSQL como caché principal.

Decisión: Implementar Redis utilizando el patrón Cache-Aside.

Justificación

- **Justificación 1: Reducción significativa de latencia**

Las consultas frecuentes pueden resolverse directamente desde memoria sin necesidad de acceder continuamente al Core Bancario.

Esto permite reducir tiempos de respuesta desde cientos de milisegundos o segundos hasta decenas de milisegundos, mejorando significativamente la experiencia del usuario.

- **Justificación 2: Disminución de carga sobre sistemas legados**

Los sistemas Core suelen ser recursos críticos y costosos. Redis permite reducir la cantidad de consultas realizadas a dichos sistemas, disminuyendo costos operativos y evitando cuellos de botella durante períodos de alta demanda.

- **Justificación 3: Control explícito sobre la estrategia de cacheo**

El patrón Cache-Aside permite que la aplicación controle cuándo almacenar, invalidar o actualizar información.

Esto proporciona mayor precisión y consistencia respecto a mecanismos automáticos de caché menos controlables.

Consecuencias Positivas

- Menor latencia.
- Menor carga sobre el Core.
- Mejor experiencia de usuario.
- Escalabilidad de lectura.

Consecuencias Negativas

- Posible inconsistencia temporal de datos.
- Necesidad de estrategias de invalidación adecuadas.

ADR-013 - CQRS + Event Sourcing Parcial + Outbox Pattern

Estado: Aceptado

Contexto:

La plataforma debe satisfacer simultáneamente requerimientos de:

- Alto volumen de consultas.
- Auditoría completa.
- Trazabilidad regulatoria.
- Integración desacoplada entre componentes.

Las necesidades de lectura son considerablemente superiores a las necesidades de escritura, especialmente para movimientos históricos y consultas de clientes.

Alternativas Evaluadas

- CRUD tradicional.
- Event Sourcing completo.
- CQRS simple.
- CQRS + Event Sourcing parcial + Outbox Pattern.

Decisión: Implementar CQRS complementado con Event Sourcing parcial para auditoría y Outbox Pattern para garantizar consistencia en la publicación de eventos.

Justificación

- **Justificación 1: Separación eficiente entre lectura y escritura**

Las operaciones de lectura y escritura poseen comportamientos distintos.

CQRS permite optimizar cada modelo de forma independiente, mejorando rendimiento y reduciendo complejidad en consultas masivas.

- **Justificación 2: Trazabilidad completa de eventos críticos**

Las entidades financieras requieren reconstruir operaciones históricas ante auditorías, investigaciones o disputas.

Event Sourcing parcial permite conservar eventos relevantes sin necesidad de implementar dicho patrón sobre todo el dominio.

- **Justificación 3: Garantía de consistencia entre base de datos y mensajería**

El Outbox Pattern evita escenarios donde una transacción es almacenada exitosamente pero el evento asociado no es publicado.

Esto resulta crítico para procesos regulatorios, auditorías y notificaciones obligatorias.

Consecuencias Positivas

- Auditoría robusta.
- Mejor rendimiento en consultas.
- Consistencia entre eventos y datos.
- Mayor desacoplamiento.

Consecuencias Negativas

- Incrementa complejidad arquitectónica.
- Requiere gobernanza de eventos.
- Necesita monitoreo adicional.

ADR-014 - Observabilidad basada en OpenTelemetry, Prometheus y Grafana

Estado: Aceptado

Contexto:

La plataforma estará compuesta por múltiples microservicios distribuidos, integraciones externas, componentes cloud y procesos asíncronos.

La detección temprana de incidentes y el análisis de problemas requieren visibilidad completa del comportamiento del sistema.

Alternativas Evaluadas

- Logs tradicionales.
- Monitoreo exclusivo de infraestructura.
- APM propietario.
- OpenTelemetry + Prometheus + Grafana.

Decisión: Adoptar una estrategia de observabilidad basada en OpenTelemetry, Prometheus, Grafana y CloudWatch.

Justificación

- **Justificación 1: Trazabilidad distribuida extremo a extremo**

OpenTelemetry permite seguir una transacción desde el canal digital hasta los sistemas Core, incluyendo llamadas REST, eventos Kafka y procesos asíncronos. Esto reduce significativamente los tiempos de diagnóstico y resolución de incidentes.

- **Justificación 2: Estandarización y neutralidad tecnológica**

OpenTelemetry se ha convertido en el estándar de facto para la observabilidad moderna. Su adopción evita dependencias excesivas con herramientas propietarias y facilita futuras migraciones.

- **Justificación 3: Monitoreo proactivo basado en métricas de negocio**

Prometheus y Grafana permiten construir dashboards y alertas orientadas a indicadores relevantes para el negocio, como:

- Transferencias procesadas.
- Errores de autenticación.
- Tiempo promedio de respuesta.
- Disponibilidad de servicios.

Consecuencias Positivas

- Diagnóstico rápido.
- Mejor operación.
- Menor MTTR.
- Mayor visibilidad del sistema.

Consecuencias Negativas

- Incrementa almacenamiento de métricas y logs.
- Requiere gobierno sobre telemetría.

ADR-015 - Kubernetes (Amazon EKS) como Plataforma de Orquestación

Estado: Aceptado

Contexto:

La arquitectura propuesta estará compuesta por múltiples microservicios con necesidades independientes de despliegue, escalamiento y recuperación.

La organización requiere una plataforma que permita automatizar operaciones, soportar crecimiento futuro y facilitar estrategias avanzadas de despliegue.

Alternativas Evaluadas

- Máquinas virtuales tradicionales.
- AWS ECS.
- Serverless completo.
- Kubernetes (Amazon EKS).

Decisión: Adoptar Kubernetes administrado mediante Amazon EKS.

Justificación

- **Justificación 1: Escalamiento automático basado en demanda**
Kubernetes permite escalar servicios individualmente según métricas de consumo, CPU, memoria o métricas de negocio.
Esto optimiza costos y garantiza disponibilidad durante picos de carga.
- **Justificación 2: Capacidades avanzadas de resiliencia y auto-healing**
Mediante Liveness Probes, Readiness Probes, ReplicaSets y mecanismos de reconciliación automática, Kubernetes puede detectar y recuperar componentes fallidos sin intervención manual.
Esto contribuye directamente al cumplimiento de los objetivos de disponibilidad definidos para la plataforma.
- **Justificación 3: Soporte para estrategias modernas de despliegue**
Kubernetes facilita la implementación de Blue-Green Deployments, Canary Releases y Rollbacks automáticos.
Estas capacidades reducen el riesgo asociado a cambios en producción y permiten entregar nuevas funcionalidades con mayor frecuencia y seguridad.

Consecuencias Positivas

- Alta disponibilidad.
- Auto-healing.
- Escalabilidad automática.
- Despliegues avanzados.
- Portabilidad tecnológica.

Consecuencias Negativas

- Curva de aprendizaje elevada.
- Mayor complejidad operativa inicial.
- Requiere madurez DevOps y observabilidad.

